

1 Overview and objectives

For this task, a grammatical evolution (GE) implementation was compared to a structure based grammatical evolution program (SBGE), for the purpose of forecasting energy demands. The output of the genetic program is a function that forecasts the expected energy load 15 minutes into the future. Individuals of varying fitness levels were produced to solve this problem.

2 Dataset and preprocessing

The initial dataset included the following fields:

- utc_timestamp:
- Electricity_load
- Residential_electricity_price
- Residential_solar_generation
- Residential_wind_generation
- Temperature
- Relative Humidity

For the task, a time-series forecasting model is required. it was decided that for forecasting purposes, only the current load, time and date, as well as a number of lag loads (previous loads leading up to that load), would be used, as well as a previous day load.

The following is an example of a given input and output row of the data.

Input row:

```
utc_timestamp,Electricity_load,Residential_electricity_price  
    ,Residential_solar_generation,Residential_wind_generation  
    ,Temperature,Relative Humidity  
31/12/2014 23:00,0.045,0,0,0,9.8,85.8
```

Output row:

```
load,load_n1,load_n2,load_n3,load_n4,load_n5,load_n6,  
    load_prev_day  
0.7808219178082192, 0.541095890410959, 0.6917808219178083,  
    0.6917808219178083, 0.5547945205479452,  
    0.6712328767123288, 0.6643835616438357,  
    0.5547945205479452
```

It is hypothesised that only the current and previous loads will turn out to be the strongest predictor of the future load, and, perhaps, the load at the same time as the previous day (which may act as some anchoring value, since load fluctuates on a daily cycle).

The Electricity_load field was processed in the following way:

Processing	Example output
Min-max scaling performed, such that each value is between 0 and 1 (the training sets min and max was used, because the test set was assumed to be unseen, however the test sets data was independently min-maxed scaled). Additionally, 6 lag loads and the previous day load were produced from this column. $t - 96$ was used for the previous day, since there are 96 15-minute intervals in a day. The following formulas were used: • $load = \frac{curr_load - min_load}{max_load - min_load}$ • $load_n1 = previousLoads.at(t - 1)$ • $load_n2 = previousLoads.at(t - 2)$ • $load_n3 = previousLoads.at(t - 3)$ • $load_n4 = previousLoads.at(t - 4)$ • $load_n5 = previousLoads.at(t - 5)$ • $load_n6 = previousLoads.at(t - 6)$ • $load_prev_day = previousLoads.at(t - 96)$	load: 0.780, load_n1: 0.541, load_n2: 0.6917, load_n3: 0.691, load_n4: 0.554, load_n5: 0.671, load_n6: 0.664, load_prev_day: 0.554

3 Terminal and function set and representation

Function set: $F = \{ +, -, *, /, \text{SQR} \}$

Terminal set: $T = \{ R, load, load_n1, load_n2, load_n3, load_n4, load_n5, load_n6, load_prev_day \}$

Range of R: $[-2, 2]$

Each individual of the population is a genome of size 10. This was to create a gene with sufficient expressiveness. During evaluation, however, wrapping may occur, such that the expression tree represented by the gene is of greater depth.

The function set included all functions that allowed for linear relationships, as well as a square function, for a parabolic function. The functions in this set are quite robust against noise, as they would not likely overfit when the data is noisy. On the other hand, including sin and cos lead to a reduction in validation MSE. This is because they may have found patterns in the noisy training data that would not provide a robust output when tested on the validation set. Thus, simpler and more robust linear operators were included, along with the square function for the case of a non-linear relationship.

The final function set included the most common operators to produce straight line and parabolic functions. This is because energy load is likely to have some straight line interaction between the previous loads and the projected load at $T + 1$. Alternatively, if the loads turn out to be parabolic, there may be a non-linear interaction whereby the square or multiplication function will turn out to be the most accurate representation. In the case of a hyperbolic function, division will allow for this representation.

The current load (*load*) and 6 lag loads (*load_n1*, *load_n2*, *load_n3*, *load_n4*, *load_n5*, *load_n6*) were included in the terminal set. 6 lag loads is likely overkill, however it was left up to the GE to discover which lag loads were the strongest predictors of future load.

The previous day load was also included, since energy fluctuates on a daily cycle (eg: high load during dinner times).

R is an ephemeral random constant, ranged [-2, 2]. The range was discovered through parameter tuning to produce the most stable results. Higher ranges usually resulted in deeper trees (bloat), since the most accurate individuals usually had to downscale higher values to a more suitable range.

The divide operator used was a protected divide. In the case that the denominator was less than and epsilon of 1e-6, the function returns 1. Returning 1 (as opposed to the numerator) acts as a neutral multiplier, preserving the pre-existing structure. This is to prevent the fitness of otherwise good trees with helpful building blocks from plummeting due to a very small denominator. It also protects the function from an illegal division by 0.

4 Initial population generation

The initial population was generated using the ramped half-and-half strategy. The reason for this, is because having trees of varying depths increases diversity in the initial population (they are spread over a wider area of the search space initially), and in the generations to come, since crossover will take place with trees of varying sizes and shapes, thus spreading out the population over the search space.

At initialisation, the production rules were mapped to a "tree" growing recursively until either the maximum depth is reached (for those grown with both full grow and grow) or until it probabilistically terminates. With the full-grow trees (half of the population), the trees were allowed to grow to the full depth. With the grow trees, the trees could terminate before reaching the maximum at each depth with a 50% chance of terminating.

The following population sizes were tested:

- 150 (75 full-grow, 75 grow)
- 300 (150 full-grow, 150 grow)
- 450 (225 full-grow, 225 grow)
- 600 (300 full-grow, 300 grow)
- 750 (375 full-grow, 375 grow)

5 Fitness function and fitness evaluation

Mean squared error was chosen over mean absolute error to penalise large errors more aggressively. The error was simply the sum of the differences squared

between the individuals output and the target output in the training, validation and test sets.

All rows in the training, validation and test sets were used.

If an individual did not terminate within a reasonable number of wraps (500), a very large value was returned, signifying an invalid gene. This would prevent the gene from being selected for the next population.

6 Noise floor of the data and target MSE

To give my GE a target mean squared error, I estimated the noise floor of the dataset. This was done by calculating the k nearest neighbour of each row in the dataset (according to all the lag loads and previous day load). The mean of these variance values represents the noise floor.

Set	Noise floor (MSE)
Training	0.007328
Validation	0.015923
Test	0.082083
Entire set	0.009723

The *highestHitError* (target MSE) parameter was used to control what the system records as accurate given the noise floor. This was calculated to be 0.012 (adding 15% to the noise floor of the entire set). It follows that if an individual produces any result within 15% of the noise floor on the test set, it counts as a hit.

7 Selection method

Tournament selection was used, since it allows one to easily control the selection pressure. This problem benefited from a low selection pressure, which is more difficult to achieve with other selection methods. Convergence to a particular equation form was likely in this particular problem, and emphasis was placed on preventing premature convergence. A small *TournamentSize* (4), along with quite a large population sizes.

8 Genetic operators

8.1 Crossover

To cross over two genes, a random crossover point was selected (from 0 to the max length). The codons from parent one up until the crossover point were copied to child one, and after that point were copied to the front of child1, and the codons from parent 2 after the crossover point were copied to child1.

Similarly, to create child 2, we select the codons before the crossover point in parent1 and the codons after the crossover point in parent2.

Only a random subset of the population was subject to crossover, which was dictated by the *CrossoverRate* parameter. The others were subject to mutation.

Crossover allows exploitation of an area of the search space by mixing genetic material from selected (fitter) parents.

8.2 Mutation

All nodes not subject to crossover, underwent mutation. For mutation a random codon in the individual was selected, and it was replaced with a new random value between 0 and 255. Mutation allows exploration of the search space by introducing new genetic material.

8.3 Reproduction (elitism)

For each generation, the fittest individual found during selection is cloned directly into a random position of the next generation to prevent that individual being lost due to crossover or mutation.

8.4 SBGE freezing mechanism

For the SBGE implementation, a cyclical solution was employed. For each cycle, different structural controls were used on two separate occasions. At the start of each cycle, a percentage of the top individuals (*freezeEliteIndividualsPercent*) had the top third of their codons frozen. This was to protect elite genetic material near the root node (mutating a codon near the top of an individual has a greater impact on the way that individual evaluates, moving it to a different part of the search space). This was to make sure that fit individuals continue to exploit their current area of the search space.

Only the top 30% of individuals were subject to freezing top nodes.

Similarly, the bottom 30% of individuals were subject to having the end of the gene frozen. This would make sure that crossover and mutation happens near the front of the gene, drastically changing the outcome, and thus moving the individual to a different part of the search space.

Frozen nodes were not subject to crossover or mutation (they could not be selected as crossover or mutation points) for all the *localSearchGenerations* generations. At the end of every cycle but the last, a percentage of the weakest individuals would be moved to a different part of the search space during the *peturbGenerations*.

In summary, elite individuals were subject to a front freeze, protecting accurate building blocks, while weak individuals were subject to a back freeze, moving the search to a new area.

9 Experimental setup

9.1 Data set

The dataset was split up as follows:

Training set: 70%

Validation set: 15%

Test set: 15%.

The data was not shuffled, meaning that the validation set tested the models ability to predict unseen data "in the future", and the test set was further "in the future" (the most recent data). This allowed forecasting to occur.

The training set was used to evolve the population, while the validation set was used to test the quality of the population and test whether the stopping condition was met (the average MSE is below *Highest Stopping Error*).

Total Population	Full-Grow per Depth	Grow per Depth
150	15	15
300	30	30
450	45	45
600	60	60
750	75	75

Table 1: Population Grow Strategies and Tree Depth Distributions

With each configuration, 5 different seeds were tested. This means that in total 5 initial population generation strategies (with varying depths and population sizes), 2 algorithms (Canonical / SBGE), averaged across 5 different seeds were tested, giving a total of 50 runs.

9.2 Parameters

9.2.1 Generations and Cycle Parameters

Local Search Generations Per Cycle

This was the number of generations that would occur on a cyclical basis before the perturbation generations execute. A value of 10 was chosen to would allow the population enough time to converge on a local optima. During this time, the SBGE applies structural freezing to the top layers of elite individuals, which promotes exploitation of the local search space.

Perturb Generations Per Cycle

After each cycle, front codons of the weakest genes were frozen, after which crossover was applied, moving these weak individuals to different parts of the

search space in hopes of finding a better local optima to exploit. A value of 2 was chosen, since this is more than sufficient to break up structures near the front.

Total Cycles

The experiment was conducted over 5 full cycles. This means that there were 5 phases of exploration and convergence for each run. On the final cycle, no perturbation is performed to preserve the fitness of the population.

9.2.2 Selection and Variation Parameters

Tournament Size

A tournament size of 3 was used. The low tournament size reduces selection pressure by maintaining diversity well into the run. This prevents premature convergence on a local optima (this would occur if the tournament size were too high, and elite individuals were much more likely to be chosen). Choosing a low selection pressure means that over many generations of selection, the fitness of the entire population will gradually increase, and the population will hopefully converge on a part of the search space that is closer to the global optima.

Crossover Rate

A crossover rate was set relatively high (70%), since the structure-based mechanics ensured diversity. This ensures that successful genetic material is used in future generations, and the local search space is exploited.

Mutation Rate

The mutation rate was set to 0.3 (30%). The remaining individuals which were not subject to crossover, undergo mutation. This mutation rate makes sure that diversity is maintained throughout the run by introducing new genetic material by replacing a random subtree.

9.2.3 Terminal and Constant Logic

Choose Constant Probability

This is the probability that a leaf node will end up being a constant as opposed to a variable node (equation input). The probability of choosing either a constant or a variable was exactly half.

Tune Constant Probability

This is the probability that a constant node will be fine-tuned during mutation. In the case that a constant node is selected as the mutation point, if a coin flip succeeds based on this probability, the value of that node is altered instead

of mutation occurring to the entire subtree. The value for this parameter is relatively high, since this problem benefitted from experimenting with many different coefficient and constant values within the equations.

Premature Leaf Probability

During initial population generation for each tree which grew with the ramped half-and-half initialisation, there was a 25% chance at each depth for a branching node to become a leaf node. This variation in depth increases diversity during the initialisation.

9.2.4 Constraint and Error Parameters

Parsimony Pressure

This was a very small penalty applied to deeper trees, ensuring that if a solution was found which had the same MSE, but a smaller tree, it would be chosen over the deeper / more bloated tree. The main goal of this parameter was to slightly aid interpretability by encouraging smaller trees as solutions.

Highest Stopping Error

After calculating the error on the entire validation set, if an individual achieved an error below this MSE, the algorithm proceeded to the test set.

Highest Hit Error

A hit is used to determine whether an individual produced satisfactory results on the validation set. The value for a hit is any error at or below 0.012 MSE, which is within 15% of the noise floor of the data.

9.2.5 SBGE Specific Parameters

Freeze Elite Individuals Percent

The elite freeze percentage was set to 0.3 (30%). This dictates that the top 30% of the population are subject to the structural freezing mechanism, where their upper layers are protected to preserve the high-level logic of the model while allowing the leaves to continue evolving.

Freeze Bottom Percent

This value was set to 0.3, meaning that the weakest individuals had all nodes past a third of their depth frozen. This means that the top two thirds of nodes were now subject to crossover and mutation.

Freeze Cutoff Depth

For each freeze, it was decided that a tree should have at least a depth of at least 2 for freezing to be effective, otherwise no nodes would be altered, defeating the purpose of freezing (which is forcing nodes to be altered on either the top or bottom of trees).

9.3 Machine specifications and implementation details

- **Model:** HP EliteBook 850 G8 Notebook PC
- **Processor:** Intel Core i7-1165G7
 - **Generation:** 11th Gen Intel Core
 - **Cores / Threads:** 4 cores / 8 threads
 - **Max Clock:** up to 4.70 GHz
- **GPU:**
 - **Graphics:** Intel Iris Xe Graphics (integrated)
 - **GPU Clock:** ~1.30 GHz
- **RAM:** 16 GB

The program was implemented using C++ 23 and compiled using gcc

10 Results

For the best, median and average MSE, SBGE and canonical GE both performed equally. This is likely because both algorithms managed to have the top half of the population converge on the same part of the search space. All seeds also performed equally, meaning that there was enough search depth and breadth to remove the randomness factor.

The area where SBGE was an improvement, was with the worst MSE. This is because the freezing mechanism aided poorer performing individuals to move to a more lucrative part of the search space by freezing nodes closer to the front of the gene, meaning that the result is changed drastically.

Table 2: Comparison of Evaluation Metrics: Canonical GE vs. Structure-Based GE (SBGE)

Metric	Stat	Canonical GE	Structure-Based GE (SBGE)
valBestMSE	Mean	0.0158067	0.0158067
	Median	0.0158067	0.0158067
	Std Dev	0.0	0.0
testBestMSE	Mean	0.0182733	0.0182733
	Median	0.0182733	0.0182733
	Std Dev	0.0	0.0
valMedianMSE	Mean	0.0158067	0.0158067
	Median	0.0158067	0.0158067
	Std Dev	0.0	0.0
testMedianMSE	Mean	0.0182733	0.0182733
	Median	0.0182733	0.0182733
	Std Dev	0.0	0.0
valWorstMSE	Mean	4.098×10^{79}	1.396×10^{22}
	Median	5.449×10^9	3.965×10^9
	Std Dev	1.419×10^{80}	4.831×10^{22}
testWorstMSE	Mean	5.543×10^{79}	1.396×10^{22}
	Median	5.135×10^9	4.401×10^9
	Std Dev	1.920×10^{80}	4.831×10^{22}
valAvgMSE	Mean	8.0×10^{79}	8.0×10^{79}
	Median	8.0×10^{79}	8.0×10^{79}
	Std Dev	8.355×10^{79}	8.355×10^{79}
testAvgMSE	Mean	1.360×10^{83}	1.360×10^{84}
	Median	1.6×10^{80}	1.6×10^{81}
	Std Dev	4.610×10^{83}	4.610×10^{84}
valStdDevMSE	Mean	1.347×10^{83}	1.346×10^{84}
	Median	8.0×10^{79}	8.0×10^{79}
	Std Dev	4.614×10^{83}	4.614×10^{84}
testStdDevMSE	Mean	1.360×10^{83}	1.360×10^{84}
	Median	1.6×10^{80}	1.6×10^{81}
	Std Dev	4.610×10^{83}	4.610×10^{84}

seed	valBestMSE	valWorstMSE	valMedianMSE	valAvgMSE	valStdDevMSE	testBestMSE	testWorstMSE	testMedianMSE	testAvgMSE	testStdDevMSE
1001300_canonical0	0.015806689	4.918435e+80	0.015806689	1.6e+80	1.6e+80	0.01827327	6.651835e+80	0.01827327	1.6e+80	1.6e+80
1001_300_SB1	0.015806689	0.015806689	0.015806689	1.6e+80	1.6e+80	0.01827327	0.01827327	0.01827327	1.6e+80	1.6e+81
1001450_canonical2	0.015806689	0.015806689	0.015806689	0.015806689	1.3877788e-17	0.01827327	0.01827327	0.01827327	0.01827327	2.0816682e-17
1001_450_SB3	0.015806689	0.015806689	0.015806689	0.015806689	1.3877788e-17	0.01827327	0.01827327	0.01827327	0.01827327	2.0816682e-17
1001600_canonical4	0.015806689	7728458.2	0.015806689	1.6e+80	1.6e+80	0.01827327	7721108.1	0.01827327	1.6e+80	1.6e+80
1001_600_SB5	0.015806689	7.930093e+09	0.015806689	1.6e+80	1.6e+81	0.01827327	8.8013304e+09	0.01827327	1.6e+81	1.6e+81
1001750_canonical6	0.015806689	0.015806689	0.015806689	1.6e+80	1.6e+82	0.01827327	0.01827327	0.01827327	1.6e+82	1.6e+82
1001_750_SB7	0.015806689	0.015806689	0.015806689	1.6e+80	1.6e+83	0.01827327	0.01827327	0.01827327	1.6e+83	1.6e+83
1002300_canonical8	0.015806689	1.6625771e+23	0.015806689	1.6e+80	1.6e+84	0.01827327	1.6626749e+23	0.01827327	1.6e+84	1.6e+84
1002_300_SB9	0.015806689	1.6737754e+23	0.015806689	1.6e+80	1.6e+85	0.01827327	1.6738735e+23	0.01827327	1.6e+85	1.6e+85
1002450_canonical10	0.015806689	0.015806689	0.015806689	0.015806689	1.3877788e-17	0.01827327	0.01827327	0.01827327	0.01827327	2.0816682e-17
1002_450_SB11	0.015806689	0.015806689	0.015806689	0.015806689	1.3877788e-17	0.01827327	0.01827327	0.01827327	0.01827327	2.0816682e-17
1002600_canonical12	0.015806689	1.7512615e+11	0.015806689	1.6e+80	1.6e+80	0.01827327	1.7519608e+11	0.01827327	1.6e+80	1.6e+80
1002_600_SB13	0.015806689	4.2273961e+10	0.015806689	1.6e+80	1.6e+80	0.01827327	4.2274929e+10	0.01827327	1.6e+81	1.6e+81
1002750_canonical14	0.015806689	0.015806689	0.015806689	0.015806689	5.5511151e-17	0.01827327	0.01827327	0.01827327	0.01827327	2.4286129e-17
1002_750_SB15	0.015806689	0.015806689	0.015806689	0.015806689	5.5511151e-17	0.01827327	0.01827327	0.01827327	0.01827327	2.4286129e-17
1003300_canonical16	0.015806689	1.3680255e+11	0.015806689	1.6e+80	1.6e+80	0.01827327	1.3680873e+11	0.01827327	1.6e+80	1.6e+80
1003_300_SB17	0.015806689	1.5668856e+20	0.015806689	1.6e+80	1.6e+80	0.01827327	1.5023225e+20	0.01827327	1.6e+81	1.6e+81
1003450_canonical18	0.015806689	971182.67	0.015806689	2189.5133	45734.428	0.01827327	987633.57	0.01827327	2227.8742	46509.463
1003_450_SB19	0.015806689	971182.67	0.015806689	2189.5132	45734.428	0.01827327	987633.57	0.01827327	2227.8742	46509.463
1003600_canonical20	0.015806689	1.0892201e+10	0.015806689	1.6e+80	1.6e+80	0.01827327	1.0263399e+10	0.01827327	1.6e+80	1.6e+80
1003_600_SB21	0.015806689	7.0654392e+12	0.015806689	1.6e+80	1.6e+80	0.01827327	7.222513e+12	0.01827327	1.6e+81	1.6e+81
1003750_canonical22	0.015806689	9.1064149e+32	0.015806689	1.6e+80	1.6e+80	0.01827327	9.9567797e+32	0.01827327	1.6e+82	1.6e+82
1003_750_SB23	0.015806689	7.0433163e+18	0.015806689	1.6e+80	1.6e+80	0.01827327	9.3335954e+18	0.01827327	1.6e+83	1.6e+83

11 Conclusion

This study compared SBGE to canonical GE for the purpose of time-series forecasting. The results show that SBGE manages to improve the performance of worse performing individuals, allowing them to move to more optimal parts of the search space.

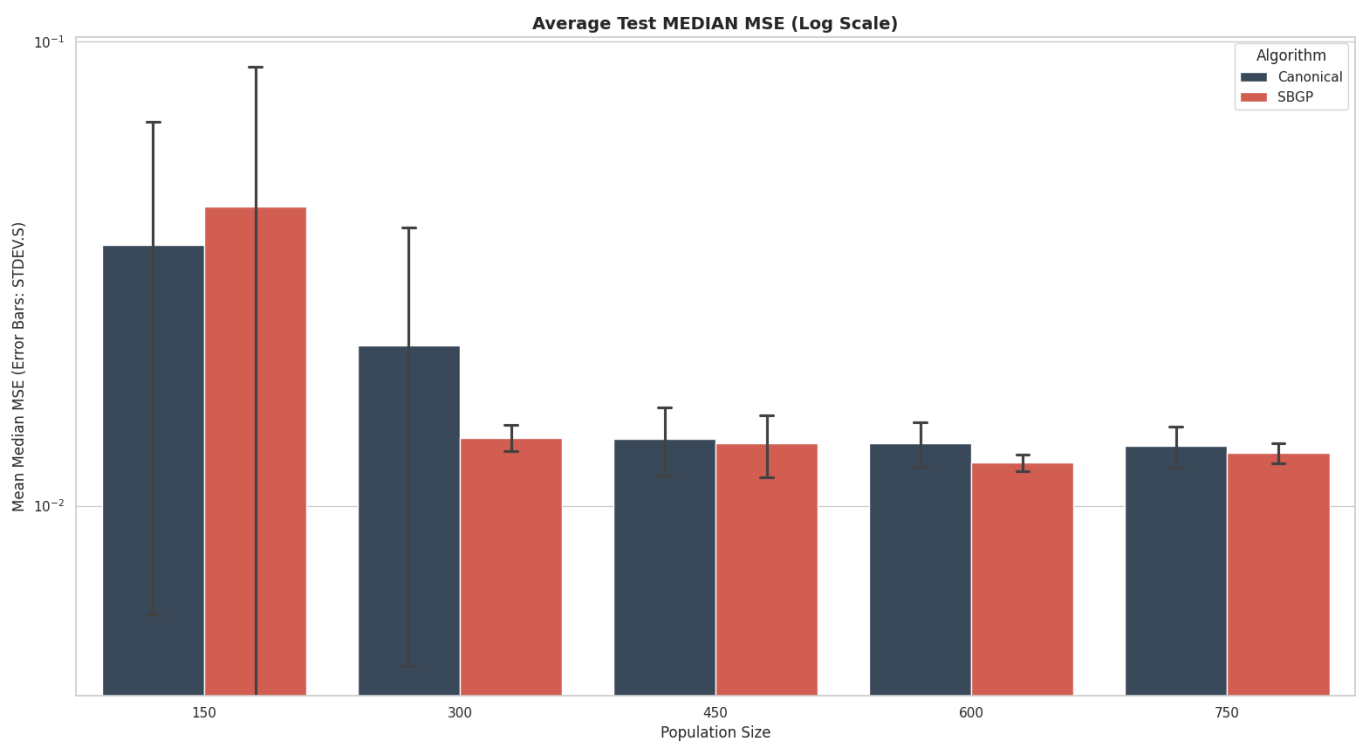


Figure 1: Median mean squared error averaged across 5 runs with different seeds for canonical and SBGE.

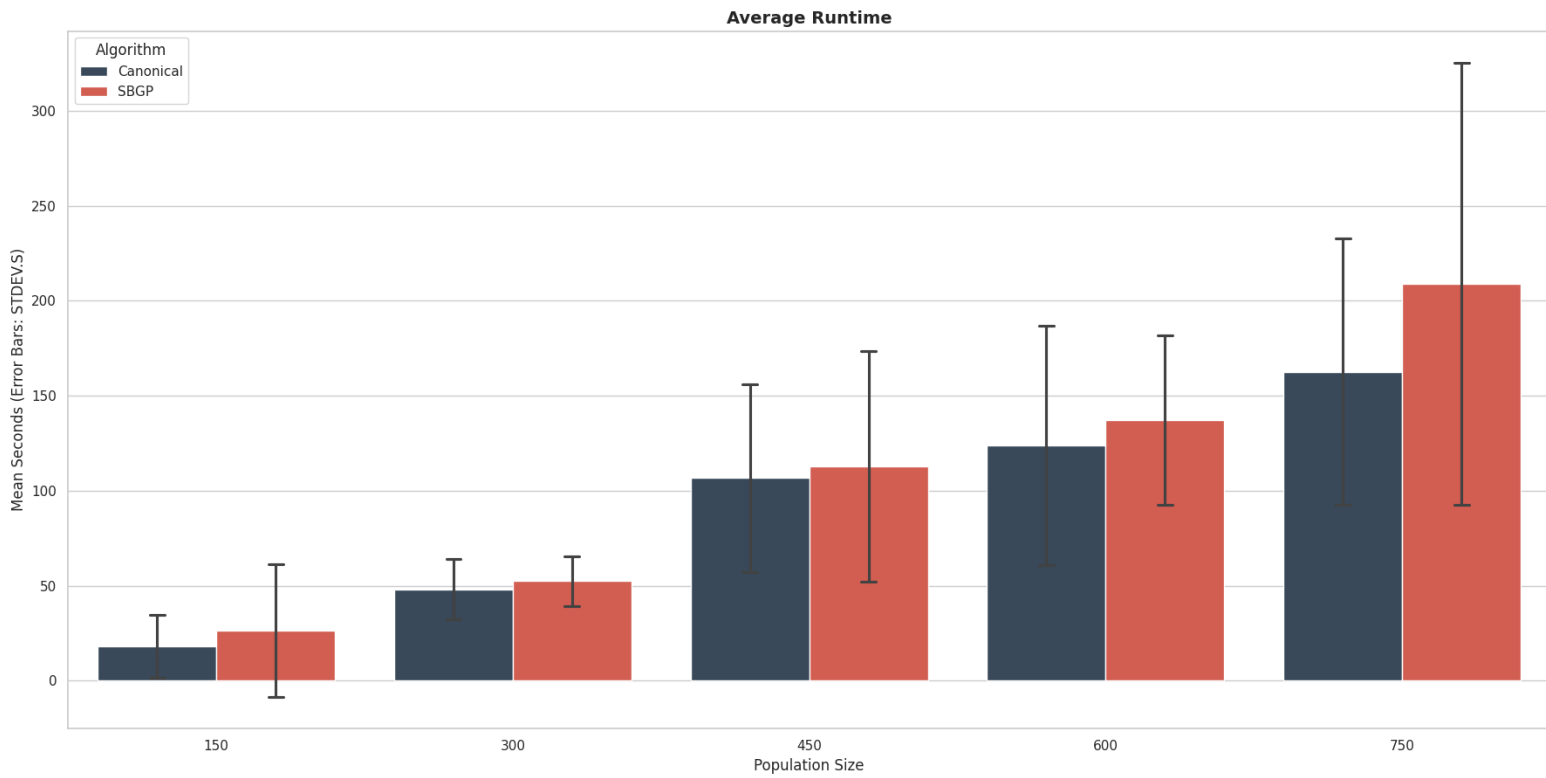


Figure 2: Run-time averaged across 5 runs with different seeds for canonical and SBGE.